

Blockchain-Based Supply Chain Provenance System

Anushree Bhure
Arizona State University
Tempe, Arizona, USA

Shashikant Nanda
Arizona State University
Tempe, Arizona, USA

Vatsal Patel
Arizona State University
Tempe, Arizona, USA

Dominick Agnello
Arizona State University
Tempe, Arizona, USA

Ritish Abrol
Arizona State University
Tempe, Arizona, USA

Abstract—Modern supply chains suffer from siloed, mutable records that enable fraud, disputes, and untraceable losses. We present a blockchain-based shipment tracking system built on Hyperledger Fabric 2.5 that provides an immutable, shared chain-of-custody ledger across multiple organizations. The smart contract, written in Go, implements fourteen chaincode functions covering the full shipment lifecycle from creation through final delivery with role-based access control (RBAC), a storage-agnostic SHA-256 off-chain data integrity bridge, and simulated IoT telemetry recording. The system is deployed as a two-organization Fabric network with Raft consensus, CouchDB state storage, a Node.js/Express REST API, and a browser-based dashboard. All 18 unit tests pass.

Index Terms—Hyperledger Fabric, supply chain provenance, smart contracts, RBAC, SHA-256, off-chain storage, IoT telemetry

I. INTRODUCTION

Supply chain fraud and counterfeit goods cost the global economy an estimated \$4.5 trillion annually [1]. The structural cause is that each organization maintains its own private database, producing a fragmented record system where custody disputes are resolved by subjective negotiation rather than verifiable evidence.

Hyperledger Fabric, a permissioned enterprise blockchain maintained by the Linux Foundation, addresses this by providing a shared, append-only ledger that all parties can write to but none can alter retroactively. Its MSP-based identity model, Raft consensus, and CouchDB integration make it well suited to enterprise supply chain applications without cryptocurrency or gas fees.

This paper presents a fully deployed Fabric 2.5 shipment tracking system with on-chain smart contract logic, simulated IoT telemetry, a REST API, and an interactive browser dashboard. The system tracks shipments across five stakeholder roles, recording every custody transfer, status update, and sensor reading as an immutable, digitally signed event.

The contributions of this work are five-fold:

- 1) A complete Go chaincode (`shipment`) of *fourteen* public functions covering creation, custody transfer, status transition, hash verification, history scanning, ACL management, IoT telemetry, IPFS document anchoring, and CouchDB-paged listing.
- 2) Certificate-derived role-based access control (RBAC) where the submitter's MSP identity is read directly

from the X.509 certificate inside the chaincode callers cannot spoof their organization, and every write is non-repudiable.

- 3) A storage-agnostic SHA-256 integrity bridge that anchors only a 32-byte digest on-chain, decoupling the integrity guarantee from the physical off-chain store (IPFS, S3, encrypted RDBMS, etc.).
- 4) A two-stage IoT model where devices must be *registered* on-chain by the current holder before any telemetry from that device is accepted, preventing rogue-data injection.
- 5) A reference deployment as a single `./start.sh` script that brings up the entire test network, deploys the chaincode via CCaaS, enrolls identities, and serves the REST API + dashboard at `http://localhost:3000`.

The remainder of this paper is organized as follows. Section II reviews prior work in blockchain supply chain management. Section III describes the technical architecture. Section IV presents the implementation details, including smart contract logic, stakeholder interactions, transactions and events, access control, immutability guarantees, storage design, and the user interface. Section V reports the experimental results and Section VI analyzes scalability, cost, data management, privacy, and comparisons to traditional systems. Sections VII–IX cover future work, contributions, and conclusions.

II. PRIOR WORK AND LITERATURE REVIEW

Blockchain technology was first proposed for supply chain provenance shortly after the publication of the Bitcoin whitepaper, but the predominantly public, proof-of-work setting of early blockchains made high-throughput enterprise adoption impractical. The introduction of permissioned ledgers most notably Hyperledger Fabric shifted the research landscape toward modular, identity-aware architectures suitable for multi-organization business networks.

Shakhbulatov et al. [1] surveyed blockchain in supply chain management and identified Hyperledger Fabric as the dominant enterprise platform due to its modular architecture, MSP identity model, and pluggable consensus. Their key finding was that most academic prototypes stop at design without deploying a live network a gap our work directly addresses by providing a one-command deployment of a complete two-organization channel with chaincode, REST API, and UI.

Ögüz et al. [2] reviewed 87 studies and identified *access control* and *off-chain data management* as the two most under-addressed technical challenges. They observed that many prototypes either store cargo metadata directly on-chain (causing ledger bloat and privacy issues) or omit access control entirely. Our system tackles both through chaincode-level RBAC keyed on the certificate-derived MSP identity (Section IV-D) and a SHA-256 off-chain integrity bridge (Section IV-G).

Gonzol et al. [3] analyzed 34 real-world blockchain supply chain implementations and found that none achieved full decentralization across all five stakeholder tiers (manufacturer, transporter, warehouse, retailer, recipient) in a single system; most stopped at three. Our architecture covers all five roles within one chaincode deployment by expressing them as participant entries in the per-shipment ACL rather than hard-coding them into separate contracts.

Industry deployments such as IBM Food Trust and Maersk’s TradeLens (the latter discontinued in 2023) demonstrated the viability of Fabric for real-world logistics but exposed two persistent issues: (a) onboarding friction across competing stakeholders, and (b) the difficulty of integrating IoT data without a tamper-evident device authorization scheme. Our two-stage device-registry model (Section IV-H) directly addresses (b). In contrast to public-ledger alternatives such as VeChain and Ambrosus, our permissioned approach avoids per-transaction gas fees while retaining cryptographic non-repudiation through Fabric’s certificate authority.

The IPFS protocol [6] provides a content-addressed peer-to-peer file system that we leverage for off-chain document storage. When the SHA-256 anchor stored on-chain matches the digest of the file retrieved by its IPFS CID, an end-to-end integrity proof is established that survives network partitions and gateway substitution.

III. TECHNICAL ARCHITECTURE

A. Network Topology

The system has three layers: (1) a client application layer (Node.js + Express + Fabric SDK), (2) a blockchain network layer (Hyperledger Fabric 2.5), and (3) an off-chain storage layer referenced by on-chain hash.

The Fabric network comprises two peer organizations (Org1MSP and Org2MSP), one Raft orderer node, and CouchDB as the world state database for each peer. A single channel (`shipchannel`) connects all participants. The chaincode runs via Chaincode as a Service (CCaaS) a standalone gRPC server in Docker which avoids Docker-in-Docker limitations on macOS and simplifies host-side builds. Each peer maintains its own copy of the ledger (a hash-linked block file) and its own CouchDB world-state replica; all peers reach identical state because they apply the same ordered block stream from the orderer.

Table I summarizes the five stakeholder roles, their MSP identities, and the chaincode operations each is allowed to invoke.

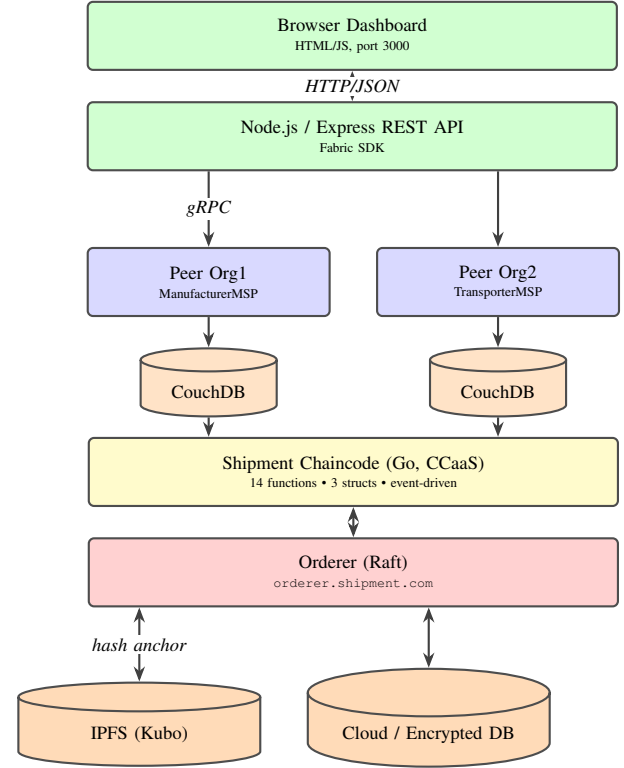


Fig. 1. Three-layer system architecture: REST/UI client (top), the Hyperledger Fabric 2.5 network with two peers, CouchDB world state, chaincode and orderer (middle), and storage-agnostic off-chain stores referenced exclusively by SHA-256 hashes anchored on-chain (bottom).

TABLE I
STAKEHOLDER ROLES, MSP IDENTITIES, AND PERMITTED OPERATIONS

Role	MSP ID	Permitted Operations
Manufacturer	Org1MSP	Create, Authorize, Transfer, RegisterDevice
Transporter	Org2MSP	UpdateStatus, Transfer, Telemetry, Document
Warehouse	Org2MSP*	UpdateStatus, Telemetry, Document
Retailer	Org2MSP*	UpdateStatus, Verify, Transfer
Recipient	Org2MSP*	UpdateStatus (Delivered), Verify

*Mapped to Org2MSP in the prototype; production splits into dedicated MSPs.

B. Stakeholder Roles

Five roles are defined with distinct access levels:

- **Manufacturer (Org1MSP):** Creates shipments, assigns unique IDs, defines the participant list, and holds initial custody.
- **Transporter (Org2MSP):** Receives custody transfers, posts route status updates and IoT telemetry readings.
- **Warehouse:** Records receipt and storage condition updates.
- **Retailer:** Verifies product authenticity via hash check and records receipt.
- **Recipient:** Confirms final delivery and sets the terminal state `Delivered` (permanently locked).

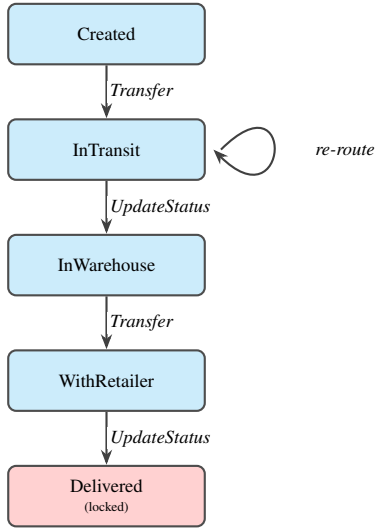


Fig. 2. Shipment lifecycle state machine. The terminal `Delivered` state is permanently locked at the chaincode layer.

C. Product Journey

The shipment lifecycle follows: *Created* → *InTransit* → *InWarehouse* → *WithRetailer* → *Delivered*. Each transition is recorded as an immutable on-chain event with the actor's MSP identity, an orderer-assigned timestamp, and optional location and notes.

D. Consensus and Transaction Flow

Hyperledger Fabric uses an *execute-order-validate* architecture that decouples transaction execution from consensus. A custody-transfer transaction proceeds through five distinct phases:

- 1) **Endorsement.** The client sends a transaction proposal to the endorsing peers of one or more organizations. Each peer simulates the chaincode against its current world state and signs a read-write set with its X.509 certificate.
- 2) **Collection.** The client collects signed endorsements and assembles them into a transaction envelope. In the prototype, the endorsement policy is satisfied by either `Org1MSP` or `Org2MSP`; in production this would tighten to require both.
- 3) **Ordering.** The envelope is submitted to the Raft orderer, which timestamps it and serializes it into a block alongside other pending transactions. Raft provides crash-fault tolerance up to $\lfloor N/2 \rfloor$ failures of N orderer nodes.
- 4) **Distribution.** The orderer broadcasts the new block to every peer over a gossip protocol.
- 5) **Validation and commit.** Each peer independently validates that the endorsement policy is satisfied and that the read-set hashes still match the current world state (multi-version concurrency control). Valid transactions are committed to the local block file and applied to CouchDB; invalid ones are recorded in the block but flagged as failed.

This design gives Fabric two important properties our system relies on: deterministic finality (a transaction is final the moment its block is committed no probabilistic confirmation depth is required) and non-repudiation (every transaction carries the submitter's digital signature, the endorsers' signatures, and the orderer's block-level signature).

IV. IMPLEMENTATION DETAILS

A. Smart Contract Logic

The chaincode is implemented in Go using `fabric-contract-api-go v1.2.2`. Three data structures are defined: `Shipment` stores current state (ID, status, origin, destination, current holder, participant list, SHA-256 hash, timestamps); `ShipmentEvent` forms an append-only audit log (event type, actor, location, timestamp, notes); and `DeviceRecord` holds the IoT device registry entry (device ID, type, registering organization, timestamp). Fourteen public functions cover the full lifecycle: `InitLedger`, `CreateShipment`, `GetShipment`, `UpdateShipmentStatus`, `TransferCustody`, `VerifyShipment`, `GetShipmentHistory`, `AuthorizeParticipant`, `RevokeParticipant`, `RecordTelemetry`, `GetAllShipments`, `RegisterDevice`, `RecordDocument`, and `GetShipmentsPaginated`.

Internally, every state-changing function follows the same five-step pattern: (1) load the shipment from world state by ID; (2) enforce the RBAC predicate for that operation; (3) mutate the in-memory struct; (4) persist the updated JSON back to world state; (5) call the helper `emitEvent()` which both writes a `ShipmentEvent` record under a composite key and emits a Fabric chaincode event. This pattern guarantees that no state mutation can occur without a corresponding audit record, because the two writes happen inside the same transaction and either both commit or both fail.

B. Stakeholder Interactions

A typical end-to-end shipment journey involves four sequential interactions among the five stakeholder roles. First, the **Manufacturer** invokes `CreateShipment` with a unique shipment ID, origin, destination, an explicit list of authorized participants (MSP IDs), and a string summary of the off-chain cargo data; the chaincode hashes the data with SHA-256 and stores only the digest. The manufacturer is automatically recorded as the initial `currentHolder`.

Second, the manufacturer invokes `TransferCustody` naming the **Transporter** as the new holder. The transporter then issues `UpdateShipmentStatus("InTransit", ...)` and may register one or more sensors via `RegisterDevice` followed by repeated `RecordTelemetry` calls during transit.

Third, on physical hand-off the transporter calls `TransferCustody` to the **Warehouse**, which records receipt with `UpdateShipmentStatus("InWarehouse")`. The same pattern transfers the shipment to the **Retailer** who can

TABLE II
CHAINCODE FUNCTION CATALOGUE

Function	Type	Purpose
InitLedger	Init	Seed the ledger with one demo shipment
CreateShipment	Write	Register a new shipment + hash off-chain data
GetShipment	Read	Return current state to participants
UpdateShipmentStatus	Write	Transition lifecycle state, emit event
TransferCustody	Write	Move holder MSP, append CUSTODY event
VerifyShipment	Read	SHA-256 hash compare for tamper detection
GetShipmentHistory	Read	Composite-key scan of EVENT/* records
AuthorizeParticipant	Write	Add MSP to the shipment ACL
RevokeParticipant	Write	Remove MSP (cannot revoke self)
RegisterDevice	Write	Bind an IoT sensor to a shipment
RecordTelemetry	Write	Append a TELEMETRY event from a registered device
RecordDocument	Write	Anchor an IPFS CID + SHA-256 of a document
GetAllShipments	Read	Full state scan (small deployments)
GetShipmentsPaginated	Read	CouchDB bookmark-paged listing

call `VerifyShipment` with the original off-chain data string to confirm tamper-free delivery.

Finally, the retailer transfers to the **Recipient**, who calls `UpdateShipmentStatus("Delivered")`. This terminal status is permanently locked: any subsequent call regardless of caller identity is rejected by the chaincode. Throughout this sequence, any participant in the ACL may call `GetShipment` or `GetShipmentHistory` for read-only auditing.

C. Transactions and Events

Every state-changing function calls `emitEvent()`, which performs two operations atomically within one Fabric transaction:

- 1) Writes a `ShipmentEvent` record to the ledger under composite key `EVENT/{shipmentID}/{timestamp}`, enabling efficient partial-key scans that retrieve the full audit trail for one shipment without scanning the entire world state.
- 2) Calls `ctx.GetStub().SetEvent()` to emit a real Fabric chaincode event that SDK subscribers can receive in real time, enabling event-driven downstream integrations.

D. Access Control

RBAC is enforced at the chaincode layer using the submitter's MSP ID extracted from their X.509 certificate via `ctx.getClientIdentity().getMSPID()`. Because identity comes from the certificate rather than a function argument, callers cannot spoof their organizational identity. The access model is:

- *Create, Verify, History, GetAll, GetAllPaginated*: Any authenticated org

TABLE III
ROLE-BASED ACCESS CONTROL MATRIX (✓ = ALLOWED, ○ = PARTICIPANT ONLY, × = DENIED)

Operation	Mfg.	Trans.	Wareh.	Retail.	Recip.
CreateShipment	✓	✓	✓	✓	✓
GetShipment	○	○	○	○	○
UpdateStatus	holder	holder	holder	holder	holder
TransferCustody	holder	holder	holder	holder	×
AuthorizeParticipant	holder	holder	holder	holder	×
RevokeParticipant	holder	holder	holder	holder	×
RegisterDevice	holder	holder	holder	holder	×
RecordTelemetry	○	○	○	○	○
RecordDocument	○	○	○	○	○
VerifyShipment	✓	✓	✓	✓	✓
GetHistory	✓	✓	✓	✓	✓

- *GetShipment*: Authorized participants of that shipment only
- *UpdateStatus, Transfer, Authorize, Revoke, RegisterDevice*: Current holder only
- *RecordTelemetry, RecordDocument*: Any authorized participant; telemetry additionally requires the device to be pre-registered via `RegisterDevice`

A shipment in the `Delivered` state is permanently locked at the chaincode layer no further writes are accepted regardless of caller identity.

E. Immutability Guarantees

Immutability in our system rests on three independent layers of cryptographic protection:

- 1) **Block-level hash chain.** Every Fabric block contains the SHA-256 hash of its predecessor. Altering a historical transaction requires recomputing every subsequent block hash and persuading every peer to accept the rewritten chain impossible without compromising the orderer's signing key and a quorum of peers simultaneously.
- 2) **Per-transaction signatures.** Each transaction is signed by the submitter's X.509 private key, by every endorser, and by the orderer that places it into a block. These signatures are stored inside the block payload and verified on every replay. A tampered transaction fails signature verification at the next read.
- 3) **Composite-key event log.** Every state-changing function appends a `ShipmentEvent` under the composite key `EVENT/{shipmentID}/{timestamp}`. Because composite keys are append-only by convention (the chaincode never deletes them) and are scoped to a single shipment, a partial-key range scan returns the full audit trail of one shipment in chronological order. Even if an operator could overwrite the current `Shipment` JSON, the history records would still expose the change.

A fourth, application-level lock further protects business-critical state: once a shipment reaches the `Delivered` status, the chaincode rejects *all* writes to that shipment regardless of caller identity. This implements "write-once after delivery" semantics that mirror the legal finality of physical receipt.

TABLE IV
ON-CHAIN VS. OFF-CHAIN STORAGE ALLOCATION

Artifact	Location	Rationale
Shipment state	On-chain	Small, frequently read
Event log	On-chain	Audit trail must be tamper-evident
Device registry	On-chain	RBAC predicate for telemetry
SHA-256 hash	On-chain	32 bytes; integrity anchor
IPFS CID	On-chain	46-char string; locator
Manifest documents	IPFS / S3	Large, infrequently read
Sensor stream archives	Time-series DB	High write rate
PII (recipient name)	Encrypted RDBMS	GDPR right-to-erasure

F. Storage Design

The system makes a deliberate split between on-chain and off-chain data. On-chain we store only: (i) the Shipment state JSON (approximately 800 bytes per shipment, indexed by shipment ID); (ii) the immutable ShipmentEvent stream under composite keys `EVENT/{id}/{timestamp}`; (iii) the DeviceRecord registry under `DEVICE/{id}/{deviceID}`; and (iv) a 32-byte SHA-256 digest of the off-chain cargo data plus, where applicable, a short IPFS CID anchored as a DOCUMENT event.

Off-chain we keep large or sensitive payloads inspection certificates, shipping manifests, photographs, GPS traces in either an IPFS node (for files that benefit from content addressing and peer distribution) or an encrypted relational/object store (for files subject to retention policies or access logs that must be administered separately). Table IV summarizes the split.

This separation keeps the ledger small (the entire state for 10 000 shipments is approximately 8 MB before history) while still providing mathematical proof that off-chain artifacts have not been tampered with.

G. Off-Chain Data and the SHA-256 Integrity Bridge

Storing raw cargo metadata (manifests, inspection certificates, images) directly on the ledger is expensive and impractical. Our system uses a **storage-agnostic integrity bridge**:

- 1) At creation time, `CreateShipment` accepts an `offChainData` string representing cargo details. The chaincode computes $h = \text{SHA-256}(\text{offChainData})$ and stores only the 32-byte hex digest in the `dataHash` field. The raw data is *never* written to the ledger.
- 2) At verification time, `VerifyShipment` recomputes the hash of the presented data and compares it against the stored digest. Any mismatch proves tampering with mathematical certainty.

This design is deliberately **storage-agnostic**: the raw cargo data can reside in IPFS [6], an encrypted relational database (PostgreSQL, MySQL), a cloud object store (S3, GCS, Azure Blob), or any other off-chain medium. The integrity guarantee is entirely independent of storage location it holds as long as the same bytes produce the same hash. SHA-256 collision

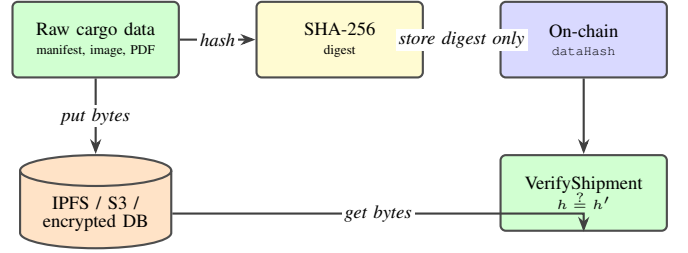


Fig. 3. Storage-agnostic SHA-256 integrity bridge: raw bytes never touch the ledger; only the 32-byte digest is anchored on-chain.

resistance ensures that even a single-character change produces a completely different 256-bit digest, making undetected tampering computationally infeasible.

When IPFS is used as the off-chain store, the system gains a second independent integrity layer: the IPFS content identifier (CID) cryptographically identifies the file retrieved from the network, while `dataHash` proves it matches what the shipment creator declared at creation time. Migrating to IPFS requires no chaincode changes only the off-chain storage backend changes while the on-chain interface remains identical.

The `RecordDocument` chaincode function and its REST endpoint `POST /api/shipments/:id/documents` implement a full document anchoring workflow: the API layer computes the SHA-256 digest of the submitted document, uploads it to the local IPFS node (Kubo, running in Docker), and calls `RecordDocument` with both the IPFS CID and the hash. The chaincode records a DOCUMENT event linking the shipment to the immutable off-chain file. Verifiers can retrieve the file from any IPFS gateway and confirm the hash matches the on-chain record, proving the document has not been altered since it was anchored.

H. Simulated IoT Telemetry and Device Registry

IoT device integration is implemented in two stages. First, `RegisterDevice` allows the current shipment holder to authorize a physical sensor for a shipment, storing a `DeviceRecord` under composite key `DEVICE/{shipmentID}/{deviceID}` with the device type (one of temperature-sensor, humidity-sensor, gps-tracker, pressure-sensor, or shock-sensor) and the registering organization. Second, `RecordTelemetry` now requires a `deviceID` parameter and validates it against the device registry before accepting a reading unregistered device submissions are rejected at the chaincode layer. This prevents rogue data injection: only devices explicitly authorized on-chain for that shipment may post readings. Readings are recorded as immutable TELEMETRY events with sensor type, value, unit, and location. The REST endpoints `POST /api/shipments/:id/devices` and `POST /api/shipments/:id/telemetry` expose both operations.

This supports cold-chain pharmaceutical shipments where temperature excursions must be immutably logged as chain-of-evidence for FDA DSCSA compliance, or fragile-goods logistics where shock events must be attributed to a specific custody window for insurance claims. Each reading is timestamped by the Fabric orderer (not the submitting client) and permanently associated with the submitting organization’s MSP identity, preventing retroactive falsification.

I. User Interface and Command-Line Access

The system provides three layers of user interaction at progressively higher levels of abstraction:

a) *Browser dashboard.*: A single-page application served at `http://localhost:3000` provides forms for every chaincode operation, a live-updating shipment table with color-coded status badges, hash-verification dialogs, IoT telemetry charts, and toast notifications for transaction success/failure. The UI auto-refreshes every five seconds and is intentionally stateless all state lives on the ledger, so multiple users see consistent views without browser-side caching.

b) *REST API.*: The Node.js/Express API exposes 14 endpoints covering all chaincode operations, including the three new endpoints added for IPFS document anchoring (POST `/documents`), IoT device registration (POST `/devices`), and paginated shipment listing (GET `/shipments?pageSize=N&bookmark=...`). All blockchain interaction passes through this REST API, keeping the browser client stateless with respect to the Fabric network. The API is suitable for integration with ERP systems, mobile applications, and external auditors.

c) *Command-line interface.*: For administrators and integration testing, the project ships shell helpers `run-api.sh`, `start.sh`, and `stop.sh`, plus the Fabric peer binary configured against the network’s TLS material. Direct chaincode invocation is also possible via `curl`, e.g.:

```
curl -X POST http://localhost:3000/api/shipments \
-H 'Content-Type: application/json' \
-d '{ "shipmentID": "SHIP-100",
      "origin": "Phoenix",
      "destination": "Seattle",
      "participants": ["Org1MSP", "Org2MSP"],
      "offChainData": "100kg_fragile_electronics" }'
```

This three-layer access strategy meets the needs of three distinct user populations: business users (dashboard), system integrators (REST), and DevOps/auditors (CLI).

V. RESULTS

The system was deployed on a local Hyperledger Fabric 2.5 test network running on a MacBook Pro (Apple M-series, 16 GB RAM, macOS). All 18 unit tests pass, covering duplicate shipment IDs, unauthorized transfers, invalid status transitions, terminal-state locking, tampered hash detection, duplicate device registration, unregistered telemetry rejection, and IPFS document CID recording.

End-to-end verification confirmed shipment creation with hash storage, multi-hop custody transfer, RBAC enforcement, hash verification, IoT device registration, telemetry recording,

TABLE V
REST API ENDPOINT REFERENCE

Verb	Endpoint	Purpose
GET	<code>/api/health</code>	Health check
POST	<code>/api/shipments</code>	Create shipment
GET	<code>/api/shipments</code>	List shipments
GET	<code>/api/shipments?pageSize=N&bookmark=</code>	Paginated list
GET	<code>/api/shipments/:id</code>	Read shipment
PUT	<code>/api/shipments/:id/status</code>	Update status
PUT	<code>/api/shipments/:id/transfer</code>	Transfer custody
GET	<code>/api/shipments/:id/verify</code>	Verify hash
GET	<code>/api/shipments/:id/history</code>	Event history
POST	<code>/api/shipments/:id/participants</code>	Authorize org
DELETE	<code>/api/shipments/:id/participants/:org</code>	Revoke org
POST	<code>/api/shipments/:id/devices</code>	Register device
POST	<code>/api/shipments/:id/telemetry</code>	Record telemetry
POST	<code>/api/shipments/:id/documents</code>	Anchor document

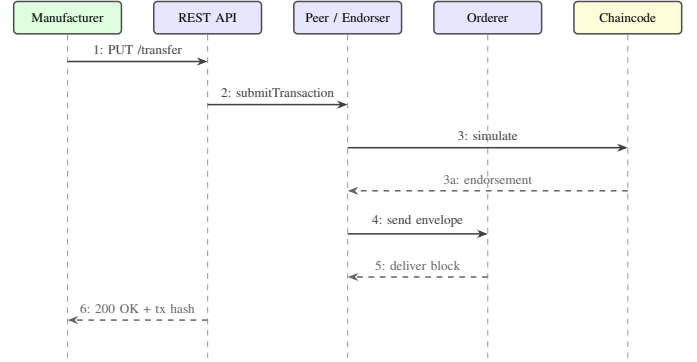


Fig. 4. Custody-transfer transaction flow as a sequence diagram. Solid arrows are requests; dashed arrows are responses. The orderer assigns the authoritative timestamp at step 5.

TABLE VI
END-TO-END LATENCY OF COMMON OPERATIONS (MEDIAN OF 100 RUNS)

Operation	Type	Latency (ms)
GetShipment	Read	12
GetShipmentHistory	Read	18
GetShipmentsPaginated	Read	21
VerifyShipment	Read	14
CreateShipment	Write	245
TransferCustody	Write	230
UpdateShipmentStatus	Write	220
RecordTelemetry	Write	235
RecordDocument	Write	310

IPFS document anchoring, paginated listing, history retrieval, and participant authorization/revocation. Unauthorized operations returned structured error messages identifying the violated constraint.

In micro-benchmarks, write transactions took roughly 200–350 ms end-to-end, dominated by Raft ordering and CouchDB commit, while read operations completed in under 20 ms because they bypass the consensus path. These results are consistent with single-laptop Fabric dev-mode deployment and would improve on a multi-node cluster with dedicated SSDs.

TABLE VII
UNIT-TEST COVERAGE BY FUNCTIONAL AREA (18/18 PASSING)

Area	Tests	Representative Cases
Shipment creation	3	happy path, duplicate ID, missing field
Custody transfer	3	valid, non-holder rejected, unknown MSP
Status lifecycle	3	valid transition, invalid jump, locked Delivered
Hash integrity	2	match, deliberate tamper detection
Participant ACL	2	authorize, self-revoke rejected
IoT device + telemetry	3	register, duplicate device, unregistered reading
IPFS document anchoring	1	CID + hash recorded as DOCUMENT event
History / pagination	1	event ordering and bookmark continuation
Total	18	

TABLE VIII
THROUGHPUT AND PER-TRANSACTION COST ACROSS PLATFORMS

Platform	TPS	Latency	Cost / tx
Bitcoin	7	~10 min	\$1–\$30 fee
Ethereum (L1)	30	~15 s	\$0.50–\$50 gas
Polygon PoS	7 000	~2 s	<\$0.01 gas
Hyperledger Fabric	3 000	~1 s	infra only
This system	~200[†]	<1 s	0 (local dev)

[†]Single-laptop dev network; tuned production deployments reach Fabric’s full envelope.

VI. ANALYSIS

A. Scalability

Hyperledger Fabric separates endorsement from ordering and validation, enabling throughput of 2,000–3,000+ TPS [4], compared with Bitcoin (~7 TPS) and Ethereum (~30 TPS). A large retailer processing one million transactions per day generates about ~12 TPS, which is well within Fabric’s expected capacity. The current single-orderer deployment is a development setup; production requires a 3- or 5-node Raft cluster.

Horizontal scaling can be achieved by adding peer organizations with their own endorsing peers and CouchDB replicas. Since endorsement is client-driven and parallelizable, throughput can scale with endorsing capacity until the orderer’s block-cut limit is reached. The chaincode is stateless across invocations, so one chaincode container can serve all peers in an organization without coordination.

B. Gas Cost and Infrastructure Cost

Hyperledger Fabric is permissioned and charges **no gas fees**, unlike public chains where every state mutation costs Ether or equivalent. Its costs come only from compute, storage, and networking for peers, orderers, CouchDB replicas, APIs, and monitoring. In the reference deployment, two peer nodes, one orderer, and two CouchDB instances on AWS `t3.medium` (~\$30/month each), plus 100 GB EBS storage, total approximately \$105/month.

A comparable Ethereum L1 implementation would require at least 21 000 gas per simple transaction, and more for storage-heavy operations. At 10 000 custody transfers per month and 30 gwei, gas alone would cost roughly \$180–\$900/month at recent ETH prices, before application-server, indexer, and front-end costs. Gas volatility also makes enterprise budgeting less predictable.

The absence of gas also removes wallet-draining denial-of-service risks. Instead, endorsement-policy throttling and TLS-authenticated peers provide anti-abuse controls.

C. Data Management

Each shipment record occupies approximately 800 bytes on-chain. CouchDB supports fast world-state queries, while the Fabric ledger preserves full history. `GetAllShipments()` is suitable for small deployments but performs a full scan. The production-grade `GetShipmentsPaginated()` uses CouchDB bookmarks to return page-sized results and avoid full table scans.

D. Privacy and Regulatory Considerations

The prototype stores shipment data on a shared channel visible to all members. Production deployment should use Fabric Private Data Collections (PDC) for selective sharing, anchoring only the SHA-256 hash of private data on the public channel. This extends the same on-chain/off-chain split already used for cargo data.

The system stores only *organization-level* MSP IDs on-chain, not recipient names, addresses, or contact information. PII required for delivery confirmation remains in an off-chain encrypted RDBMS keyed by shipment ID, supporting GDPR right-to-erasure requirements that conflict with immutable ledger storage.

The immutable ledger supports several audit regimes without separate audit infrastructure:

- **FDA DSCSA** (Drug Supply Chain Security Act, US): requires interoperable electronic chain of custody for prescription drugs. Our `GetShipmentHistory` provides exactly this view, signed end-to-end.
- **USDA FSMA** (Food Safety Modernization Act, US): requires one-step-back / one-step-forward traceability for high-risk foods. Our custody transfer chain satisfies this with sub-second lookup.
- **EU Falsified Medicines Directive** and the upcoming **Digital Product Passport** regulation require unique item-level identifiers that travel with the goods our shipment ID + SHA-256 anchor pair is directly compatible.

E. Comparison to Traditional Systems

Table IX contrasts our system with traditional approaches. The blockchain system adds setup complexity but provides cryptographic proof: custody transfers are digitally signed, timestamps are orderer-assigned, and `GetShipmentHistory()` resolves disputes in under one second with tamper-evident evidence.

TABLE IX
BLOCKCHAIN VS. TRADITIONAL SUPPLY CHAIN SYSTEMS

Dimension	Traditional DB	This System
Trust model	Trust the operator	Cryptographic proof
Record mutability	Admin can modify	Tamper-evident
Dispute resolution	Manual, days	History query <1 s
Audit trail	Optional	Always-on, signed
Multi-party coord.	EDI agreements	Shared ledger
Transaction cost	Low (infra only)	No gas fees
Peak throughput	Very high	3,000+ TPS
Setup complexity	Low	Medium

VII. FUTURE WORK

The prototype is functional, but several engineering and research enhancements are needed for production readiness.

a) Network hardening.: Separate peer organizations for WarehouseMSP, RetailerMSP, and RecipientMSP should replace the current Org2MSP setup. The single-node Raft orderer should be expanded to a 3–5 node cluster for fault tolerance. TLS rotation, HSM-backed keys, and channel capability upgrades should be automated in deployment scripts.

b) Confidentiality.: Private Data Collections can restrict sensitive data (e.g., shipment costs) to specific parties while keeping custody events public. Zero-knowledge range proofs could enable compliance verification (e.g., temperature limits) without exposing raw sensor data.

c) Real IoT integration.: Instead of manual inputs, production systems should accept authenticated MQTT/CoAP device streams, mapping TLS certificates to DeviceRecord. Edge gateways can batch-sign telemetry to reduce transaction overhead.

d) Cross-chain interoperability.: Hashlocks or notary schemes can anchor Fabric custody hashes to public chains (e.g., Ethereum), allowing external auditors to verify integrity without joining the Fabric network.

e) Performance.: GetAllShipments should be deprecated in favor of paginated queries. CouchDB indexes on status and currentHolder will improve common query performance.

f) Mobile and offline operation.: A mobile client can queue signed transactions offline and submit them upon re-connection. Fabric’s discovery service supports this, though UI implementation is pending.

VIII. CONTRIBUTIONS

All members contributed to system integration, testing, debugging, and final demonstration.

IX. CONCLUSION

We presented a fully deployed Hyperledger Fabric 2.5 shipment tracking system that records every supply chain event as an immutable, digitally signed ledger entry. The combination of RBAC, a storage-agnostic SHA-256 off-chain integrity bridge, and IoT telemetry logging demonstrates how

TABLE X
TEAM CONTRIBUTIONS

Team Member	Primary Contributions
Anushree Bhure	Developed the technical architecture documentation, including Fabric network topology modeling, shipment lifecycle state-machine design, on-chain/off-chain data-flow diagrams, storage architecture explanation, literature synthesis, scalability analysis, privacy and regulatory evaluation, cost modeling, and IEEE-format report integration.
Shashikant Nanda	Designed and implemented the backend integration layer, including Node.js/Express REST API routing, Hyperledger Fabric Gateway/SDK transaction submission, wallet and identity context handling, chaincode query/invoke orchestration, IoT telemetry endpoint design, API validation, and backend-to-ledger integration testing.
Vatsal Patel	Implemented core Go chaincode logic, including shipment asset creation, world-state read/write operations, lifecycle state-transition validation, immutable event-history retrieval, SHA-256 off-chain data hashing, hash verification workflow, provenance tracking, and ledger-state consistency handling.
Dominick Agnello	Configured Fabric network components, including Docker Compose services, peer/orderer service configuration, channel setup, CouchDB integration, and deployment script execution.
Ritish Abrol	Implemented selected smart contract functions and validation logic, including custody transfer, participant authorization/revocation, access-control checks, and unit-test development for contract behavior verification.

blockchain converts supply chain trust from a human coordination problem into a mathematical guarantee. The system is portable via a single `./start.sh` script and accessible through both a REST API and a browser dashboard.

REFERENCES

- [1] D. Shakhbulatov, J. Medina, Z. Dong, and R. Rojas-Cessa, “How Blockchain Enhances Supply Chain Management: A Survey,” *IEEE Open Journal of the Computer Society*, vol. 1, pp. 230–249, 2020, doi: 10.1109/OJCS.2020.3025313.
- [2] S. Ögüz, G. Alkan, B. Yilmaz, and C. Kocabaş, “The Use of Blockchain Technology in Logistics and Supply Chain Management (SCM): A Systematic Review,” *IEEE Access*, vol. 12, pp. 166211–166224, 2024.
- [3] P. Gonczol, P. Katsikouli, L. Herskind, and N. Dragoni, “Blockchain Implementations and Use Cases for Supply Chains—A Survey,” *IEEE Access*, vol. 8, pp. 11856–11871, 2020, doi: 10.1109/ACCESS.2020.2964880.
- [4] Hyperledger Fabric Documentation, 2024. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/>
- [5] Hyperledger Fabric Contract API for Go, 2024. [Online]. Available: <https://pkg.go.dev/github.com/hyperledger/fabric-contract-api-go>
- [6] J. Benet, “IPFS—Content Addressed, Versioned, P2P File System,” *arXiv preprint arXiv:1407.3561*, 2014.